

Lesson 6 - Quick Reference Card

□ Creating Models

```
# Simple Model
php artisan make:model Product

# With Migration
php artisan make:model Product -m

# Everything
php artisan make:model Product -mfsc
# -m: migration, -f: factory, -s: seeder, -c: controller
```

□ Retrieving Data

```
// All
$products = Product::all();
$products = Product::all(['name', 'price']);

// By key
$product = Product::find(1);
$products = Product::find([1, 2, 3]);
$product = Product::findOrFail(1);

// Where
$products = Product::where('price', '>', 100)->get();
$products = Product::where('active', 1)->get();
$products = Product::whereIn('id', [1, 2, 3])->get();
$products = Product::whereBetween('price', [100, 500])->get();

// First
$product = Product::first();
$product = Product::where('slug', 'laptop')->first();
$product = Product::firstOrFail();

// firstOrCreate
$product = Product::firstOrCreate(
    ['slug' => 'laptop'],
    ['name' => 'Laptop', 'price' => 5000]
```

```
);

// Ordering
$products = Product::orderBy('price', 'desc')->get();
$products = Product::latest()->get();
$products = Product::oldest()->get();

// Limiting
$products = Product::take(10)->get();
$products = Product::skip(10)->take(5)->get();

// Counting
$count = Product::count();
$max = Product::max('price');
$min = Product::min('price');
$sum = Product::sum('price');
$avg = Product::avg('price');
```

■ Creating Records

```
// create()
$product = Product::create([
    'name' => 'Laptop',
    'price' => 5000,
]);

// save()
$product = new Product;
$product->name = 'Laptop';
$product->price = 5000;
$product->save();

// insert (multiple)
Product::insert([
    ['name' => 'Product 1', 'price' => 100],
    ['name' => 'Product 2', 'price' => 200],
]);

// updateOrCreate
$product = Product::updateOrCreate(
    ['slug' => 'laptop'],
    ['name' => 'Gaming Laptop', 'price' => 7000]
```

```
);
```

Updating Records

```
// save()
$product = Product::find(1);
$product->price = 6000;
$product->save();

// update()
$product = Product::find(1);
$product->update(['price' => 6000]);

// Update multiple
Product::where('category', 'Electronics')
    ->update(['discount' => 10]);

// increment / decrement
$product->increment('views');
$product->increment('views', 5);
$product->decrement('stock');
$product->decrement('stock', 2);
```

Deleting Records

```
// delete()
$product = Product::find(1);
$product->delete();

// destroy()
Product::destroy(1);
Product::destroy([1, 2, 3]);

// Delete with condition
Product::where('active', 0)->delete();

// Delete all
```

```
Product::truncate();
```

❑ Soft Deletes

```
// In Migration
$table->softDeletes();

// In Model
use Illuminate\Database\Eloquent\SoftDeletes;

class Product extends Model
{
    use SoftDeletes;
}

// Usage
$product->delete(); // Soft delete

// With trashed
$products = Product::withTrashed()->get();

// Only trashed
$products = Product::onlyTrashed()->get();

// Restore
$product = Product::withTrashed()->find(1);
$product->restore();

// Force delete
$product->forceDelete();

// Check
if ($product->trashed()) {
    // Deleted
}
```

▣ Query Scopes

```
// Local Scope in Model
public function scopeActive($query)
{
    return $query->where('active', 1);
}

public function scopeCheap($query, $maxPrice = 100)
{
    return $query->where('price', '<=', $maxPrice);
}

// Usage
$products = Product::active()->get();
$products = Product::active()->cheap(50)->get();

// Global Scope
protected static function booted()
{
    static::addGlobalScope('active', function ($query) {
        $query->where('active', 1);
    });
}

// Ignore Global Scope
$products = Product::withoutGlobalScope('active')->get();
```

▣ Accessors & Mutators

```
use Illuminate\Database\Eloquent\Casts\Attribute;

// Accessor (Read)
protected function name(): Attribute
{
    return Attribute::make(
        get: fn ($value) => ucfirst($value),
    );
}

// Mutator (Write)
protected function name(): Attribute
```

```

{
  return Attribute::make(
    set: fn ($value) => strtolower($value),
  );
}

// Both
protected function price(): Attribute
{
  return Attribute::make(
    get: fn ($value) => number_format($value, 2) . ' SAR',
    set: fn ($value) => (float) str_replace(',', '', $value),
  );
}

// Usage
$product = Product::find(1);
echo $product->name; // Goes through Accessor
$product->name = 'LAPTOP'; // Goes through Mutator

```

□ Mass Assignment

```

// In Model
class Product extends Model
{
  // Allow
  protected $fillable = [
    'name',
    'price',
    'stock',
  ];

  // Or block
  protected $guarded = [
    'id',
    'created_at',
    'updated_at',
  ];

  // Allow all (unsafe!)
  protected $guarded = [];
}

```

```
}
```

Model Properties

```
class Product extends Model
{
    // Table name
    protected $table = 'products';

    // Primary key
    protected $primaryKey = 'id';
    protected $keyType = 'int';
    public $incrementing = true;

    // Timestamps
    public $timestamps = true;
    const CREATED_AT = 'created_at';
    const UPDATED_AT = 'updated_at';

    // Fillable / Guarded
    protected $fillable = ['name', 'price'];
    protected $guarded = ['id'];

    // Casts
    protected $casts = [
        'price' => 'float',
        'is_active' => 'boolean',
        'published_at' => 'datetime',
    ];
}
```

📅 Timestamps

```
// Enable/Disable
public $timestamps = true; // Default
public $timestamps = false; // Disable

// Customize
```

```
const CREATED_AT = 'creation_date';
const UPDATED_AT = 'last_update';

// Save without updating
$product->saveQuietly();
$product->timestamps = false;
$product->save();

// Update Timestamps
$product->touch();
```

▣ Chunking

```
// For large datasets
Product::chunk(100, function ($products) {
    foreach ($products as $product) {
        // Process
    }
});

// Lazy Loading
Product::lazy()->each(function ($product) {
    // Process
});
```

▣ Quick Examples

```
// Create product
$product = Product::create([
    'name' => 'Laptop',
    'price' => 5000,
    'stock' => 10,
]);

// Show active cheap products
$products = Product::where('active', 1)
    ->where('price', '<', 1000)
    ->orderBy('name')
```



```
        ->get();

// Update stock
$product = Product::find(1);
$product->decrement('stock');
$product->increment('sales');

// Delete old products
Product::where('stock', 0)
    ->where('updated_at', '<', now()->subMonths(6))
    ->delete();

// Popular products
$popular = Product::where('views', '>', 1000)
    ->orderBy('views', 'desc')
    ->take(10)
    ->get();
```

▣ Best Practices

- ▣ Use Eloquent instead of raw SQL
- ▣ Use Query Scopes for reusability
- ▣ Always define \$fillable or \$guarded
- ▣ Use Soft Deletes for safe deletion
- ▣ Use chunk() for large datasets
- ▣ Don't use all() for large datasets
- ▣ Don't forget fillable when using create()
- ▣ Don't use \$guarded = [] in production

▣ Quick Links

- [Main Lesson](#)
- [Previous Lesson](#)
- [Next Lesson](#)